

数据	组织	
	移动	i 将 i 设置为 9
循环	调试 (十六进制长 (i))	'debug, output i in hex
	条件	i,#loop 如果 i 递减且不为 -1，则循环。
	跳转	#\$ 别乱跑，待在这里。
i	-	预留一个注册表作为 'i'

运行 Ctrl-F10 后，会打开调试窗口，如下所示：

```
Cog0 初始化 $0000_0000 $0000_0000 加载
Cog0 i —
Cog0 i —
Cog0 i —
Cog0 i —
Cog0 i —
Cog0 i —
Cog0 i — 4
Cog0 i —
Cog0 i —
Cog0 i —
Cog0 i —
```

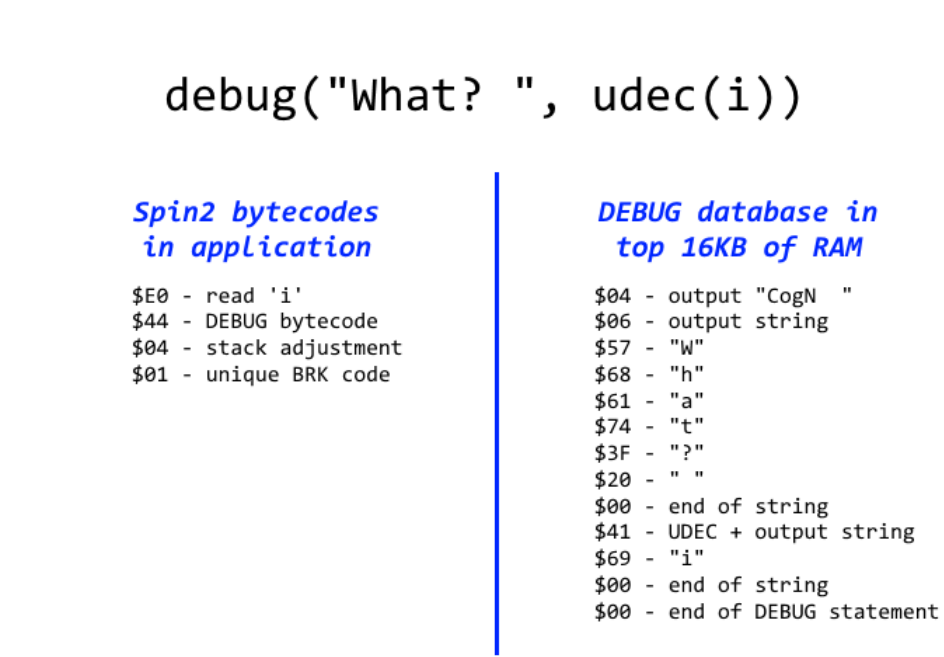
在第一个标题 报告中的一行，您会看到 Cog0 从 \$00000 加载我们的 PASM 程序。之后，程序运行，您会看到 'i' 从 9 递减迭代。0.
“调试内存使用”中
如果在 MOV 指令中将 “9” 改为 “99” 并希望减慢速度，请在 DEBUG 语句中添加一个 DLY 指令，并确保将毫秒数表示为 #250，因为一个普通的 250 会被解释为寄存器 250：

调试 (十六进制长 (i), dly(#250)) ' 调试，报告后输出 i 的十六进制值并延迟 250 毫秒

调试 内存利用率

Here is what the memory utilization looks like for 一个 Spin2 调试语句。您可以在 Spin2 侧看到，读取变量 'i' 需要字节码，然后是三个强制性的
bytecodes make up the actual DEBUG instruction.

栈调整字节指示解释器将栈向下移动多远，以便有效地 “弹出” 为调试事件准备的所有表达式。在这种 “i” 的情况下，栈只需要向下移动四个字节（一个长）。当调用调试器时，调试器所需的值将排列在当前的 Spin2 栈指针正上方。



The 'unique BRK code' byte is used as an ind查找 DEBUG 数据库中位于顶部的特定记录。emory, from which the debugger reads its commands.

In the case where DEBUG is active, but a cog has had its debug interrupt disabled via the DEBUG_ENABLE symbol, Spin2 DEBUG instructions will not trigger a debug interrupt, but they do still pop any DEBUG-intended values from the stack, so these are harmless events.

对于 PASM DEBUG 语句，在 DEBUG 命令插入的位置插入一个 'BRK #code' 指令，所有相关数据都存储在 DEBUG 数据库中。如果齿轮的 debug 中断已禁用，'BRK #code' 指令将不起作用，消耗两个时钟。

调试和中断

在调试语句期间接收到的中断请求将在调试完成后执行，但响应时间可能严重失调，导致中断的重新触发设置无法正确进行。高频周期性智能引脚中断尤其容易出现此问题。假设你在正常的 ISR（中断服务例程）中执行一个 AKPIN 指令，以降低 INA/INB 信号，使智能引脚再次升高，从而触发一个新的中断。与此同时，在 AKPIN 和 RETIx 之间，智能引脚触发，使 INA/INB 升高。这仅仅是因为你的时序周期由于调试语句而失调。由于 ISR 忙碌时发生的中断不会被看到，这会导致中断停止循环。然而，CT 中断不太容易出现此问题，因为它们有 8000_0000 个时钟周期来被识别。为了解决智能引脚重新触发问题，你可以触发 INA/IIN 高，而不是 INA/IIN 上升，但这可能会导致你的智能引脚配置出现性能问题。

解决此调试 / 中断困境的一种安全方法是，只对后台未执行中断服务例程的齿轮执行调试语句。如果中断服务例程能够容忍时间偏差，并且没有挂起中断循环的风险，则可以在了解中断时间退化的情况下执行调试语句。

图形化调试显示

调试消息可以调用内置到工具中的特殊图形化调试显示。这些图形化显示每个都以一个独特的窗口形式呈现。一旦实例化，显示就可以持续接收数据以生成动态可视化。这些显示对于开发和调试非常方便，因为各种数据类型可以在其原生上下文中查看。最多可以同时运行 32 个图形化显示。

当调试消息包含反引号 (‘) 字符 (ASCII \$60) 时，包含从反引号到消息末尾的所有内容的字符串，将被发送到图形调试显示解析器。解析器查找多种不同的元素类型，并将所有逗号视为空白：

元素类型	DEBUG memory utilizationGraphical DEBUG Displays	描述
显示类型	逻辑，示波器，绘图，位图。	这是您想要实例化 Graphical 调试显示类型的正式名称。
未知符号	MyLogicDisplay	每个图形调试显示实例必须指定一个唯一的符号名称。
instance_name	MyLogicDisplay	实例化后，一个图形化的调试显示实例通过其符号名称进行引用。
keyword	标题，POS, 大小，样本	关键字用于配置显示。它们后可能跟数字、字符串和其他关键字。
number	1024, \$FF, %1010	数字可以用十进制、十六进制（\$）和二进制（%）表示。
字符串	这是一个字符串	字符串用单引号表示。

在我们深入了解所有这些如何配合使用之前，我们需要回顾一些可以用于显示器的特殊调试显示语法。当调试语句中的第一个字符是反引号时，就会调用此语法。这会导致调试语句中的所有内容都视为字符串，除非后跟的反引号充当“转义”字符，以允许正常的或缩短的调试命令。

调试语句 (v=100, BYTTE[a]=1,2,3,4,5)	调试 Message 输出	注意
调试 (" ` 逻辑 MyDisplay 样本 ", SDEC_(v))	逻辑 MyDisplay 样本 100	常规调试语法可以驱动调试显示，但并非最佳。
调试 (LOGIC MyDisplay 样本 10)	逻辑 MyDisplay 样本 100	调试 - 显示语法更简单，输出中省略了 'CogN'。
调试 (LOGIC MyDisplay 样本 (v))	逻辑 MyDisplay 样本 100	十进制数使用 `(value) 符号输出。是 SDEC_ 的缩写。
调试 (LOGIC MyDisplay 样本 i)	逻辑 MyDisplay 样本 \$64	十六进制数使用 `\$(value) 符号输出。是 UHEX_ 的缩写。
调试 (LOGIC MyDisplay 样本 %(v))	LOGIC MyDisplay 样本 <10> 左对齐 </10>	二进制数使用 `%(value) 符号输出。UBIN_ 的缩写。
调试 ` 逻辑 MyDisplay 标题 `` #(v)'	逻辑 MyDisplay 标题 'd'	字符使用 `#(value) 符号输出。
调试 (MyDisplay UDEC_BYTE_ARRAY_(@a,5))	MyDisplay 1, 2, 3, 4, 5	常规调试命令也可以后跟反引号。

使用图形化调试显示有两步：第一步，它们必须被实例化；第二步，它们必须被输入：

使用显示：	1st	第二	第三	第四	注意
首先，实例化它。	`	显示类型	未知符号	关键词 (s), 数字 (s), 字符串 (s)	未知符号 变为 instance_name。
调试内存使用图形调试显示	`	实例名称 (s)	关键词 (s), 数字 (s), 字符串 (s)		多个显示器可以接收相同的数据。

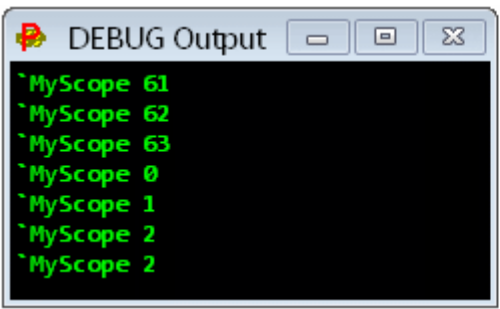
为了将所有内容结合起来，让我们在示波器上显示一个锯齿波。lay:

```
CON _clkfreq = 10_000_000

PUB go() | i

    debug(`SCOPE MyScope SIZE 254 84 SAMPLES 128)
    debug(`MyScope 'Sawtooth' 0 63 64 10 %1111)

    repeat
        debug(`MyScope `(i & 63))
        i++
        waitms(50)
```



在上面的示例中，有一个名为 MyScope 的 SCOPE 实例，其尺寸为 254 x 84 像素，并显示了 128 个样本。选择宽度为 254，因为样本编号为 0..127，我希望它们以恒定的两像素间距（127 * 2 = 254）分布。选择高度为 84，以便在波形上方和下方各有 10 个像素，波形的高度为 64 像素。定义了一个名为“锯齿波”的通道，其定义目的为显示，底部值为 0，顶部值为 63，在指定范围内有 64 像素高，并且在范围的底部向上突出 10 个像素。%11 启用顶部和底部图例值以及顶部和底部线。在 REPEAAT 块中，SCOPE 接收一个重复的 0..63 模式，该模式形成锯齿波。SCOPE 每次接收到值时都会更新其显示。如果定义了八个通道而不是一个，它将在每接收到第八个值时更新显示，绘制所有八个通道。

Currently, the following g图形调试显示已实现，但将添加更多。 e future:

显示类型	描述
逻辑	带有单位和多位标签的逻辑分析仪，可触发 1..32 个通道上的模式。
示波器	带有 1..8 个通道的示波器，可以在具有滞后效应的电平上触发。
示波器_XY	带有 1..8 个通道的 XY 示波器，持续时间为 0..512 个样本，极坐标模式，对数刻度模式
快速傅里叶变换	使用 1..8 个通道、 4..2048 个点的快速傅里叶变换，窗口化结果，对数刻度模式
频谱	包含 4..2048 点 FFT、窗口化结果、相位着色和对数刻度模式的光谱仪
绘图	通用绘图仪，支持笛卡尔和极坐标模式
终端	文本终端，大小为 300 x 200 字符，字号为 6-200 点，支持 4 种同时颜色方案。
位图	位图，1..2048 x 1..2048 像素，1/2/4/8/16/32 位图，包含 19 种颜色系统，15 种方向 / 自动滚屏模式，独立 X 和 Y 像素大小为 1..256
MIDI	带 1..128 键、力度显示、可变屏幕比例的钢琴键盘

Following are elaborations of each DEBUG display type.

LOGIC 显示逻辑分析仪，具有单位和多位标签，1..32 通道，可以按模式触发。

```
CON_clkfreq 1_000_000

PUB go() | i

    调试 ` 逻辑 MyLogic 样本 32 '低 '3 '中 '2 '高 '
    调试 ` MyLogic 触发 $07 $04 休止时间 2

    重复
        调试 MyLogic i & 63
        —
        等待 (25)
```

逻辑 实例化	描述	默认
标题 'string'	将窗口标题设置为“字符串”。	无
POS 左上角	设置窗口位置。	0, 0
样本 4_to_2048	设置要跟踪和显示的样本数。	32
间距 2_to_32	设置样本间距。显示器的宽度为 SAMPLES * 间距。	8
速率 1_to_2048	在每次显示更新之前，设置样本数（或触发数，如果已启用）。	1
线条大小 1_到_7	设置线条粗细。	1
文本大小 6_to_200	设置图例文本大小。文本高度决定逻辑电平的高度。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景和网格颜色。	黑色，灰色 4
名称 {1_to_32 { 颜色 }}	设置第一个 / 下一个通道或组名称，可选位计数，可选颜色 *。	默认颜色
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
逻辑喂养	描述	默认
触发 掩码匹配 样本偏移量	当 (数据 & 掩码) 匹配时触发 = 。如果掩码 = 0，则触发被禁用。	0, 1, 样本 / 2
休止时间 2 到 2048	设置从触发到触发所需的最小样本数。	样本
数据	数值数据以 LSB 优先的方式应用于通道。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / 绿色 / 青色 / 红 ED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

LOGIC 显示可用于显示以高速捕获的数据。在下面的示例中，P2 通过智能引脚以 333 Mbaud 速率生成 8-N-1 串行数据。流式传输器可以捕获此位流。每经过一个时钟周期，流式传输器都会记录智能引脚的 IN 信号及其从相邻引脚读取的输出状态。每当它获得四个两位样本集时，它执行 RFBYTE 操作以将其保存到枢纽 RAM 中，从而形成连续的字节、单词和 LONGS。通过调用 LONGS_2BIT 打包数据模式，我们可以让 LOGIC 显示从 LONGS 中解包两位样本集，从而产生每个 LONGS 16 个样本。

```
CON_clkfreq 1_333_333      'go really fast, 3ns clock period
  rxpin      = 24          奇偶引脚
  txpin      —      +1    奇数引脚
  样本       = 32          16 倍样本
  缓冲长度 样本 / 16      每个长字包含 16 个 2 位样本
  x          —            流式传输器模式
  mode

VAR 缓冲缓冲长度

PUB go() | i, 缓冲区 addr

    调试 ( ` 逻辑 串行样本 `( 样本 ) 间距 12 'TX' 'IN' LONGS_2BIT ) 调试 ( ` 串行触发 十 十 22 )

    buffaddr := @buff

    重复
        org
        wrpin      — ,#rxpin          rxpin 输入 txpin at rxpin+1

        wrpin      正常              设置 txpin 的异步发送模式
        wxpin      — -1, txpin        将波特率设置为 =sysclock/1, 大小 =8。
        dirh       # txpin            启用智能引脚

        wrfast     #0, 缓冲区地址      设置缓冲区为快速写模式
        xmode,#0                    开始捕获 2-bit 样本

        i,#txpin                    传输字节

        等待 wifi                    等待流捕获完成
        调试内存利用率

    调试 (v1) 串行 (v2) 十六进制长数组 _(@buff, 缓冲长度 )
    —
    等待 (20)
```

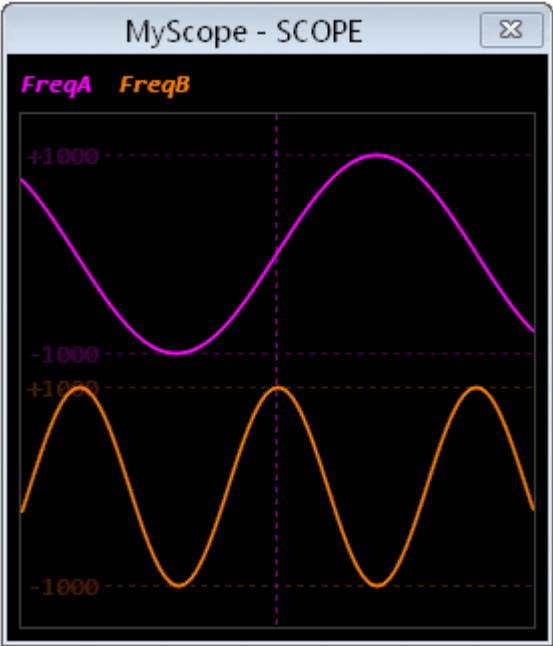

示波器 显示 示波器，具有 1..8 个通道，可以对具有滞后 电平的信号进行触发。

```
CON _clkfreq  = 100_000_000

PUB go() | a, af, b, bf

    debug(`SCOPE MyScope)
    debug(`MyScope 'FreqA' -1000 1000 100 136 15 MAGENTA)
    debug(`MyScope 'FreqB' -1000 1000 100 20 15 ORANGE)
    debug(`MyScope TRIGGER 0 HOLDOFF 2)

    repeat
        a := qsin(1000, af++, 200)
        b := qsin(1000, bf++, 99)
        debug(`MyScope `(a,b))
        waitus(200)
```

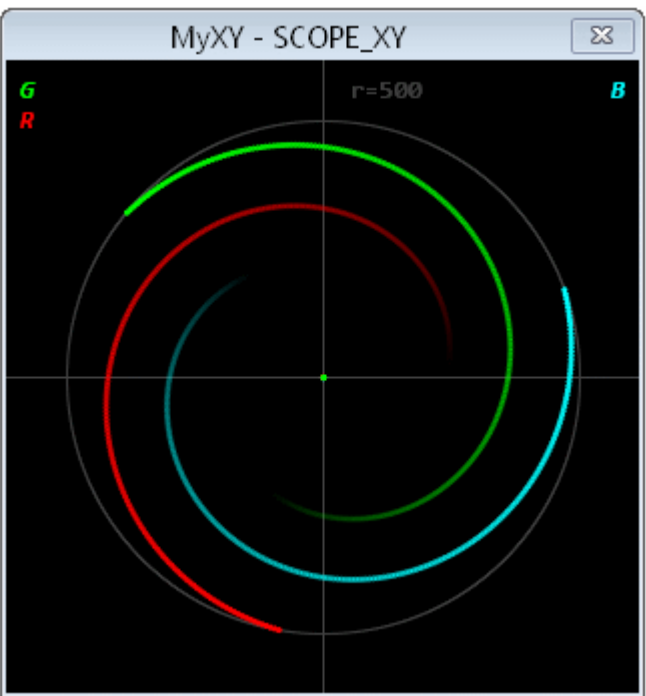


示波器 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	无
POS 左上角	设置窗口位置。	0, 0
大小 宽度高度	设置显示尺寸 (32..2048 x 32..2048)	255, 256
样本 16_to_2048	设置要跟踪和显示的样本数。	256
速率 1_to_2048	在每次显示更新之前，设置样本数（或触发数，如果已启用）。	—
点大小 0_到_32	设置像素大小以显示精确的样本点。	0
线条大小 0_to_32	将连接样本点的线条粗细设置为半像素。	3
文本大小 6_to_200	设置图例文本大小。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景色和网格颜色。	黑色，灰色 4
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	<none>
示波器 Feeding	描述	默认
名称 {min {max {y 尺寸 {y 基准 { 图例 { 颜色 } } } } }	设置第一个 / 下一个通道名称、最小值、最大值、 y 尺寸、 y 基准、图例和颜色 *。图例为 %abcd，其中 %a 到 %d 启用最大图例、最小图例、最大线、最小线。	完整，无图例，默认颜色
触发 通道 {arm_level {trigger_level { 偏移量 } } }	设置触发通道、臂级、触发级别和右偏移量。如果通道 1 禁用。	—, —, 0, 宽度 / 2
休止时间 2 到 2048	设置从触发到触发所需的最小样本数。	样本
数据	数值数据按升序应用于通道。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

SCOPE_XY Display 带有 1..8 个通道的 XY 示波器，持续时间为 1..512 个样本，极坐标模式，对数刻度模式

```
调试 = 100000000 PUB go() | i debug(` SCOPE_XY MyXY 范围
500 极坐标 360 'G' 'R' 'B') 重复 重复 i 从 0 到 500 debug(` MyXY `
(i, i, i, i+120, i, i+240)) 等待 (5)
```



SCOPE_XY 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	无
POS 左上角	设置窗口位置。	0, 0
大小 半径	设置显示半径为像素。	128
1_to_7FFFFFFF 范围	为接收数据设置单位圆半径	7FFFFFFF
样本 0_to_512	设置要跟踪和显示的样本数量，并设置持续时间。使用 0 表示无限持续时间。	256
速率 1_to_512	在每次显示更新之前设置样本数。	—
点大小 2_to_20	将样本点的点的大小设置为半像素。	调试内存利用率
文本大小 6_to_200	设置图例文本大小。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景色和网格颜色。	黑色，灰色 4
极坐标 {twopi { 偏移量 }}	设置极坐标模式、 twopi 值和偏移量。对于 twopi 值为 100000000 或 -100000000 的情况，使用 0 或 -1。	十
对数刻度	将对数刻度模式设置为放大单位圆内的点。	关闭
名称 { 颜色 }	设置第一个 / 下一个通道名称，并可选地为其分配颜色。	默认颜色
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
示波器 _XY 馈送	描述	默认
x y	X-Y 数据对按升序应用于通道。在极坐标模式下， x= 长度和 y= 角度。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

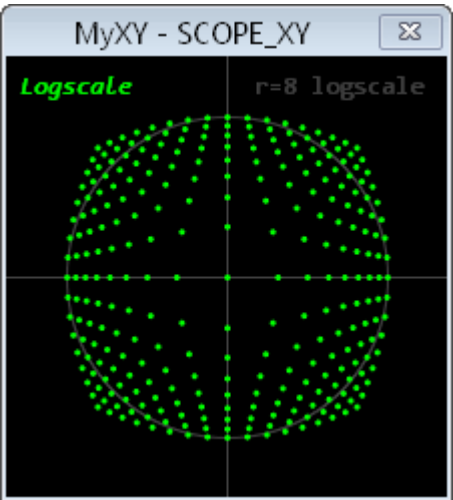
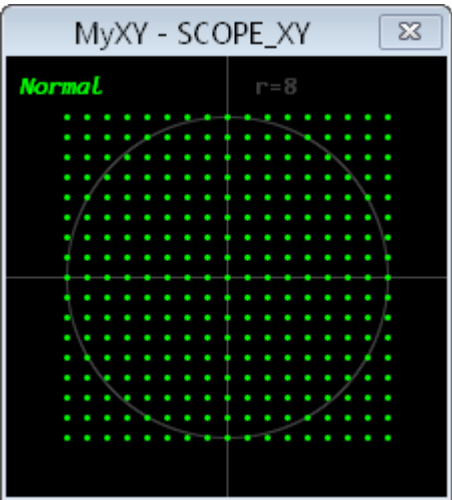
颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

```
CON_clkfreq 1_000_000      正常模式

调试 PUB go() | x, y 示波器
(` SCOPE_XY                    MyXY SIZE 80 RANGE 8 SAMPLES 0 'Normal')
  重复 x 从 -8 到 8 重复 y 从 -
  8 到 8 调试 (` MyXY `(x,y))
```

```
CON_clkfreq 1_000_000      LOGSCALE 模式放大低电平细节

调试 PUB go() | x, y 示波器
(` SCOPE_XY                    MyXY SIZE 80 RANGE 8 SAMPLES 0 LOGSCALE 'Logscale')
  重复 x 从 -8 到 8 重复 y 从 -
  8 到 8 调试 (` MyXY `(x,y))
```



快速傅里叶变换 包含 1..8 个通道、 4..2048 个点、窗口化结果、对数刻度模式的快速傅里叶变换
显示

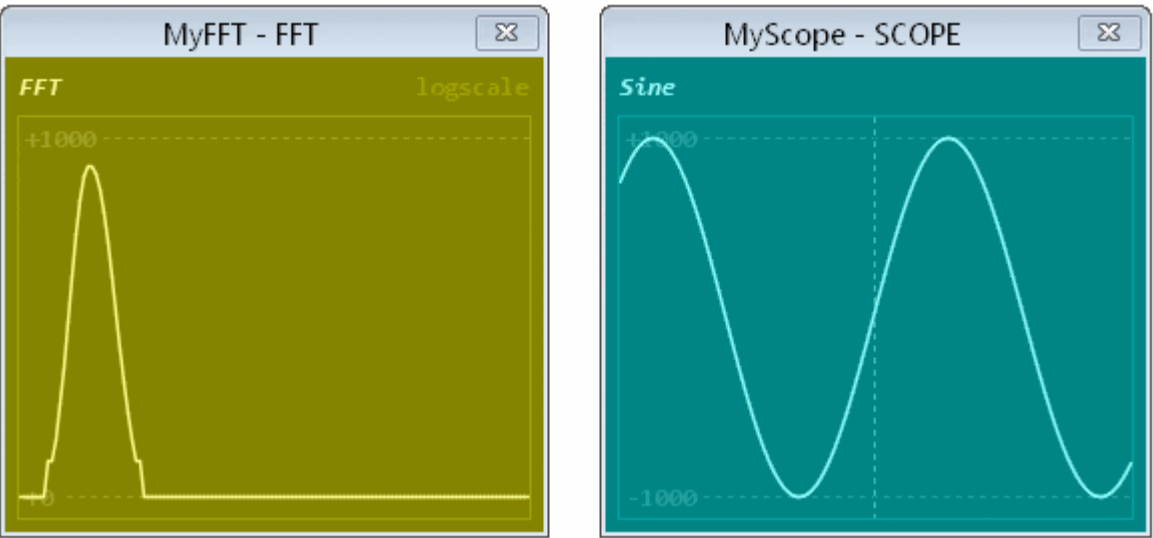
```
CON _clkfreq = 100_000_000

PUB go() | i, j, k

  ' Set up FFT
  debug(`FFT MyFFT SIZE 250 200 SAMPLES 2048 0 127 RATE 256 LOGSCALE COLOR YELLOW 4 YELLOW 5)
  debug(`MyFFT 'FFT' 0 1000 180 10 15 YELLOW 12)

  ' Set up SCOPE
  debug(`scope MyScope POS 300 0 SIZE 255 200 COLOR CYAN 4 CYAN 5)
  debug(`MyScope 'Sine' -1000 1000 180 10 15 CYAN 12)
  debug(`MyScope TRIGGER 0)

  repeat
    j += 1550 + qsin(1300, i++, 31_000)
    k := qsin(1000, j, 50_000)
    debug(`MyFFT MyScope `(k))
    waitus(100)
```



快速傅里叶变换 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	无
POS 左上角	设置窗口位置。	0, 0
大小 宽度高度	设置显示尺寸 (32..2048 x 32..2048)	256, 256
样本 4_to_2048 {first {last}}	设置 2n 个 FFT 输入点，加上第一个和最后一个结果值以显示。	512, 0, 255
速率 1_to_2048	在每次显示更新之前设置样本数。	样本
点大小 0_到_32	设置像素大小以显示精确的样本点。	0
线条大小 neg32_to_32	将连接样本点的线条粗细设置为半像素。负线条粗细将产生孤立的垂直线。	3
文本大小 6_to_200	设置图例文本大小。	编辑器文本大小
颜色 背景颜色 网格颜色	设置背景色和网格颜色。	黑色，灰色 4
对数刻度	将对数刻度模式设置为放大低级结果。	关闭
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
FFT 馈送	描述	默认
名称 { 放大倍数 { 最大 {y 尺寸 {y 基准 { 图例 { 颜色 } }	设置第一个 / 下一个通道名称、放大因子（ 2n， n = 0..11 ），最大振幅、 y 尺寸、 y 基准、图例，以及颜色 *。图例是 %abcd，其中 %a 到 %d 启用最大图例、最小图例、最大线、最小线。	完整，无图例，默认颜色
数据	数值数据被输入到通道的滑动汉宁窗中，然后 FFT 计算功率水平。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 rgb24 value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

SPECTRO 显示 配备 4 到 2048 点 FFT、相位着色和对数刻度模式的光谱仪

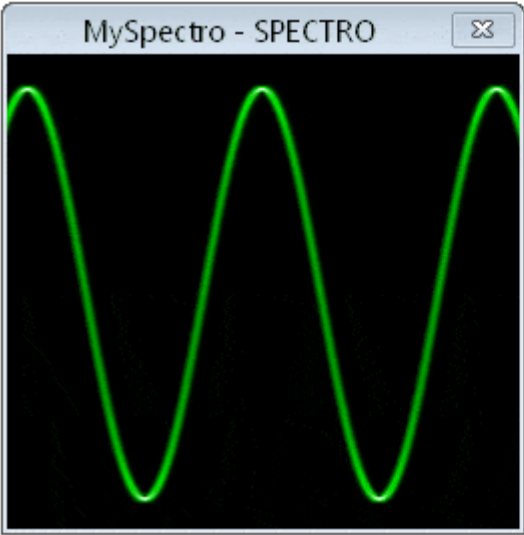
```
CON _clkfreq = 100_000_000

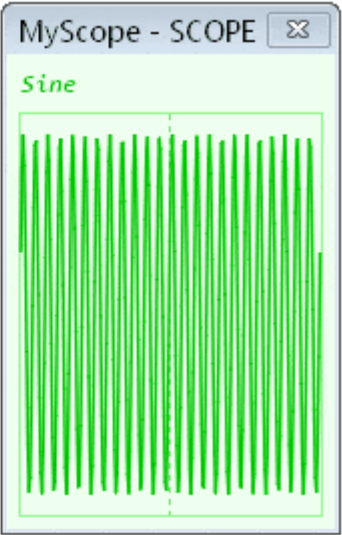
PUB go() | i, j, k

  ' Set up SPECTRO
  debug(`SPECTRO MySpectro SAMPLES 2048 0 236 RANGE 1000 LUMA8X GREEN)

  ' Set up SCOPE
  debug(`SCOPE MyScope POS 280 SIZE 150 200 COLOR GREEN 15 GREEN 12)
  debug(`MyScope 'Sine' -1000 1000 180 10 0 GREEN 6)
  debug(`MyScope TRIGGER 0)

  repeat
    j += 2850 + qsin(2500, i++, 30_000)
    k := qsin(1000, j, 50_000)
    debug(`MySpectro MyScope `(k))
    waitus(100)
```

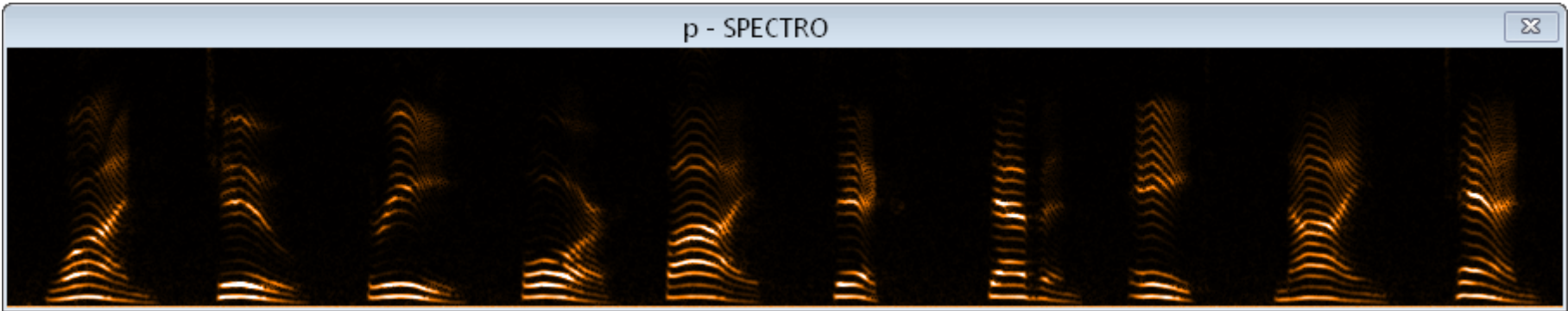




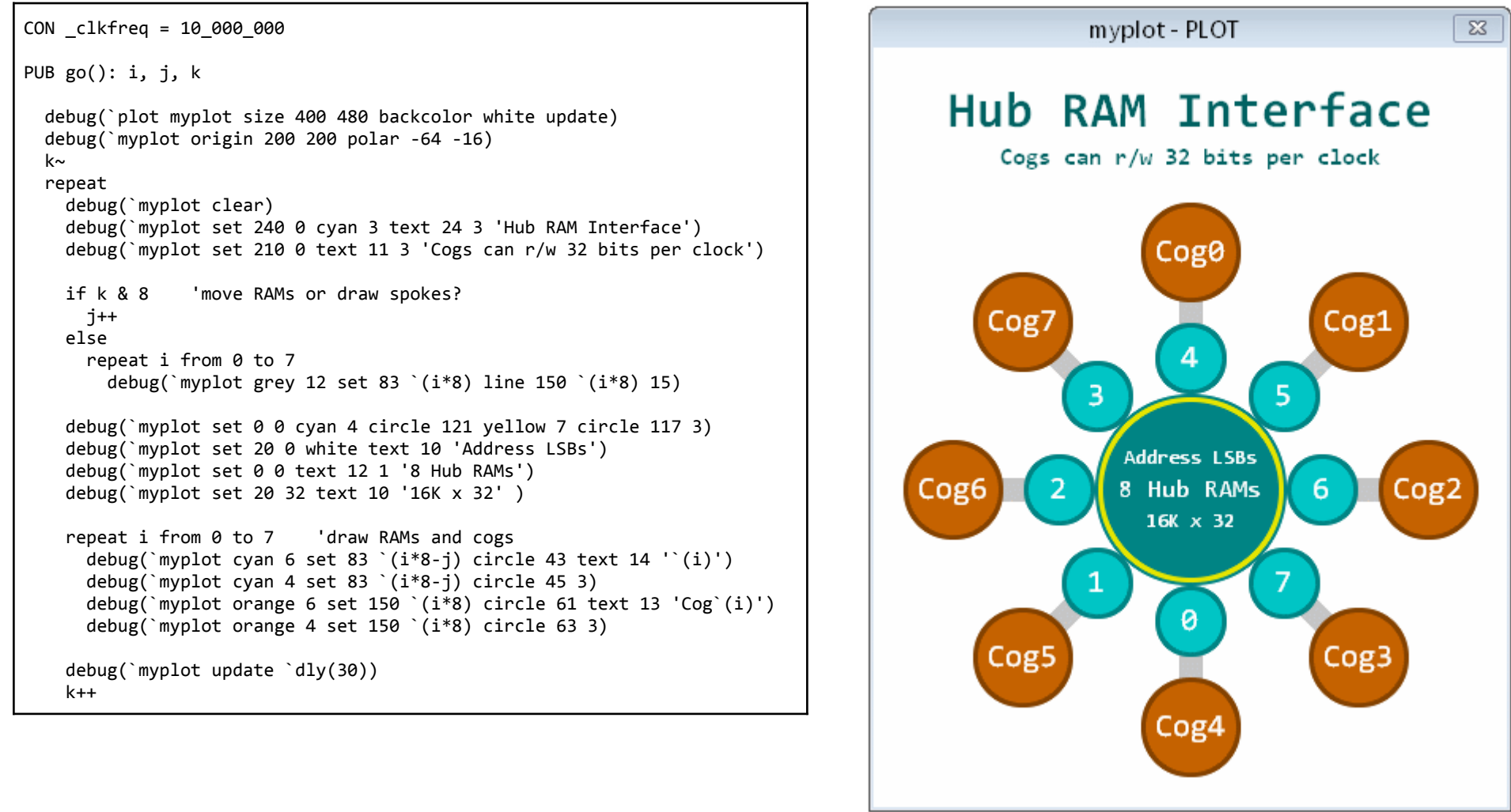
频谱 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	无
POS 左上角	设置窗口位置。	0, 0
样本 4_to_2048 {first {last}}	设置 2n 个 FFT 输入点，加上第一和最后一个结果值以显示（定义显示高度）。	512, 0, 255
深度 1_to_2048	设置要显示的垂直线 FFT 结果数量（定义显示宽度）。	256
放大倍数 0_to_11	设置放大因子 (2n, n = 0..11)。	0
范围 饱和和功率	设置像素亮度饱和的功率水平。	7FFFFFFF
速率 1_to_2048	在每次显示更新之前设置样本数。	样本 / 8
追踪 0_to_15	设置追踪模式（请参阅位图显示中的追踪动画）。	15 (右, 上, 滚动)
点大小 宽度和高度 {height}	设置光谱仪的像素宽度和像素高度，或者独立设置它们 dently.	一, 一
亮度或 HSV { 颜色或相位 }	将颜色方案设置为 Luma8(W)(X) 颜色，或使用 0..255 范围内的 HSV16(W)(X) 颜色方案相位着色偏移量。	LUMA8X 橙色
对数刻度	将对数刻度模式设置为放大低级结果。	关闭
打包数据模式	启用打包数据模式。请参阅本节末尾的描述。	无
SPECTRO 喂养	描述	默认
数据	数值数据被输入到滑动汉宁窗中，然后进行 FFT 计算功率和相位。	
清除	清除采样缓冲区和显示，等待新数据。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色 i s ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY.

以下，一个 SPECTRO 显示器接收来自麦克风引脚的 ADC 样本。这在频谱上看起来就像口头数数 “1” 到 “10” 一样。 “1” 在左侧， “10” 在右侧。水平趋势线之间的垂直距离是声门频率。较大的亮度趋势是语音表音元。这表明了我们的耳朵如何感知语音：



绘图 显示通用绘图仪，支持笛卡尔和极坐标模式



绘图 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	<none>
POS 左上角	设置窗口位置。	0, 0
大小 宽度高度	设置显示宽度（ 32..2048 ）和高度（ 32..2048 ）。	256, 256
点大小 宽度和高度 {height}	Set the display pixel-width and pixel-height together, or set them independently.	—, 1
lut1_to_rgb24	设置颜色模式。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式，请使用 RGB24 颜色加载 LUT。使用 HEX_LONG_ARRAY_ 加载颜色。	默认颜色 0..7
背景颜色	根据当前颜色模式设置背景颜色。	黑色
更新	设置更新模式。仅在接收到 “ 更新 ” 命令时，显示才会更新。	自动更新
图层馈送	描述	默认
lut1_to_rgb24	设置颜色模式。	RGB24
LUTCOLORS rgb24 rgb24 ...	对于 LUT1..LUT8 颜色模式，请使用 RGB24 颜色加载 LUT，并使用 HEX_LONG_ARRAY_ 加载值。	默认颜色 0..7
背景颜色	根据当前颜色模式设置背景颜色。	黑色
颜色 color	根据当前颜色模式设置绘图颜色。在 TEXT 之前使用以更改文本颜色。*	青色
黑白或橙色 / 蓝色 / 绿色 / 青色 / 红色 / 品红色 / 黄色 / 灰色 { 亮度 }	设置橙色到灰色颜色（默认值为 8 ）的绘制颜色和可选的 0..15 亮度。	青色
透明度 电平	设置 DOT 、 LINE 、 CIRCLE 、 OVAL 、 BOX 和 OBOX 绘制的透明度级别。0. 255 = 清除 .. 透明。	255
精确	切换精确模式，其中线条粗细以及 DOT 和 LINE 的 (x,y) 坐标以像素的 256 分之几表示。	禁用
线条大小	为 DOT 和 LINE 绘制设置线条粗细（像素）。	—
原点 {x_pos y_pos}	将原点设置为笛卡尔坐标系 (x_pos, y_pos)，如果没有指定值，则设置为当前坐标系 (x, y)。	0, 0
x y	将绘图位置设置为 (x, y)。LINE 之后，终点将成为新的绘图位置。	
点 { 线条大小 { 透明度 } }	在当前位置绘制一个带有可选线条大小和透明度覆盖的点。	
线 x y { 线条大小 { 透明度 } }	从当前位置绘制一条线到 (x,y)，可选择覆盖线条大小和透明度。	
圆直径 { 线条大小 { 透明度 } }	在可选线条粗细（无 /0 = 实线）和透明度覆盖的情况下，在当前位置绘制一个圆。	
椭圆宽度高度 { 线条大小 { 透明度 } }	在当前位置绘制一个带可选线条粗细（无 /0 = 实线）和透明度覆盖的椭圆。	
框 宽度高度 { 线条大小 { 透明度 } }	在当前位置绘制一个可选线条粗细（无 /0 = 实线）和透明度覆盖的框。	
圆角框 宽度高度 x_ 半径 y_ 半径 线条大小 { 透明度 }	在当前位置绘制一个圆角框，框的宽度、高度、 x 和 y 半径以及可选的线条粗细。（none/0 = solid）和 透明度 覆盖。	
文本大小	设置文本大小 (6..200)。	十
字体样式 style_YYXXUIWW	设置文本样式为 %YYXXUIWW： %YY 垂直对齐：%00 = 居中，%10 = 底部，%11 = 顶部。	居中

	水平对齐：居中，右对齐，左对齐。 下划线：1 = 下划线。 * 是斜体：%1 = 斜体。 %WW 是粗细：%00 = 细，%01 = 正常，%10 = 粗，以及 %11 = 粗重。	
文本角度 角度	设置文本角度。在笛卡尔模式下，角度以度为单位。	0
文本 { 大小 { 样式 { 角度 }} 'text'	绘制带有大小、样式和角度覆盖的文本。要更改文本颜色，请在 TEXT 之前声明一个颜色。	
极坐标 {twopi { 偏移量 }}	设置极坐标模式、twopi 值和偏移量。例如，POLAR -12 -3 类似于钟表。 对于两个 pi 值 100000000 或 -100000000，使用 0 或 -1。 在极坐标模式下，(x, y) 坐标被解释为 (长度 , 角度)。	十
笛卡尔	启用笛卡尔模式。这是默认模式。	
清除	清除绘图，恢复背景颜色。	
更新	更新窗口，显示当前绘图。在更新模式下使用。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

TERM Display

终端显示文本

调试 = 10000000 CON go() | i 调试(` 终端
MyTerm 大小 9 1 文本大小 40) 重复 调试(`
MyTerm 1 'Temp = `(i)' 等待 (500)

MyTerm - TERM

Temp = 58

终端 实例化	描述	默认
标题 'string'	将窗口标题设置为 “ 字符串 ”。	无
POS 左上角	设置窗口位置。	0, 0
大小 列 行	设置终端列 (1..256) 和终端行 (1..256) 的数量。	4, 20
文本大小	设置终端文本大小 (6..200)。	编辑器文本大小
颜色 文本颜色 背景颜色 ...	设置文本颜色和背景颜色组合 #0. #3. *	默认颜色
背景颜色	设置显示背景颜色。	黑色
更新	设置更新模式。仅在接收到 “ 更新 ” 命令时，显示才会更新。	自动更新
终端输入	描述	默认
字符	清除终端显示和主屏幕光标。 主页光标。 将列设置为下一个字符值。 将行设置为下一个字符值。 选择颜色组合 #0。 选择颜色组合 #1。 选择颜色组合 #2。 选择颜色组合 #3。 退格。 跳转到下一个第 8 列。 对于 13 或 10 = 换行。 32. 一 . 可打印字符。	
字符串	打印字符串。	
清除	清除显示为背景颜色。	
更新	更新窗口，显示当前文本屏幕。在更新模式下使用。	
保存 { 窗口 } ' 文件名 '	保存整个窗口或仅显示区域的位图文件 (.bmp)。	
关闭	关闭窗口。	

颜色是 a modal value, else BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).